

1. Page File

```
from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import Select


class PageName:

    def __init__(self, driver):

        self.driver = driver

        self.element_name = (By.Locator_Type, "locator_value")

    def load(self):

        self.driver.get("URL")

    def action_name(self):

        self.driver.find_element(*self.element_name).click()

    def action_with_input(self, input_value):

        self.driver.find_element(*self.element_name).send_keys(input_value)

    def get_something(self):

        return self.driver.find_element(*self.element_name).text

    def select_option(self, value):

        Select(self.driver.find_element(*self.dropdown_element)).select_by_value(value)

    def get_safely(self):

        try:

            return self.driver.find_element(*self.element_name).text

        except:

            return "default_value"

    def get_attribute_value(self):
```

```
return self.driver.find_element(*self.element_name).get_attribute("attribute_name")
```

2. Browser Setup + Cleanup — pick ONE way per file

Way 1 — inside each test:

```
driver = webdriver.Chrome()

...

driver.quit()
```

Way 2 — once, in `conftest.py`, shared by all tests:

```
import pytest

from selenium import webdriver

@pytest.fixture
def driver():
    d = webdriver.Chrome()

    yield d

    d.quit()
```

3. Test File — matching each Way

Using Way 1:

```
from selenium import webdriver

from page.page_file_name import PageName

import time

def test_case_name():
    driver = webdriver.Chrome()

    page_object = PageName(driver)

    page_object.action_name()

    page_object.action_with_input("value")

    time.sleep(2)

    assert "expected_result" in page_object.get_something()
```

```
driver.quit()
```

Using Way 2:

```
from page.page_file_name import PageName
```

```
import time
```

```
def test_case_name(driver):  
    page_object = PageName(driver)  
    page_object.action_name()  
    page_object.action_with_input("value")  
    time.sleep(2)  
    assert "expected_result" in page_object.get_something()
```

4. pytest.ini (only if using markers)

```
[pytest]
```

```
markers =
```

```
    marker_name_1: description of what this group covers
```

```
    marker_name_2: description of what this group covers
```

5. Optional Add-ons (stack above any test function)

```
@pytest.mark.marker_name
```

```
@pytest.mark.skip(reason="skip_reason")
```

```
@pytest.mark.parametrize("input_name", [value1, value2, value3])
```

6. Optional Line (inside any test body)

```
driver.save_screenshot("filename.png")
```

Placeholder

PageName
element_name
LOCATOR_TYPE
locator_value
URL
action_name
action_with_input
get_something
dropdown_element
value
default_value
attribute_name
page_file_name
test_case_name
page_object
"value"
"expected_result"
marker_name_1, marker_name_2
description of what this group covers
marker_name
skip_reason
input_name
value1, value2, value3
"filename.png"

Example

LoginPage
username_field
By.ID
"user-name"
"https://www.saucedemo.com/"
click_continue
enter_username
get_error_text
sort_dropdown
"lohi"
"0"
"src"
saucedemo_page
test_valid_login
login_page
"standard_user"
"inventory.html"
smoke, regression
"quick critical tests"
smoke
"Feature not ready yet"
username
"standard_user", "problem_user"
"login_result.png"